
VINUM

Levantamento de Requisitos

Requisitos funcionais e não funcionais da plataforma de rastreabilidade e catálogo de vinhos

Versão de referência: vnum-v1.3

Data do levantamento: 05/09/2026

1. Identificação e objetivo

Este documento registra o levantamento de requisitos da aplicação VINUM, considerando o comportamento implementado, as regras discutidas para o sistema e a estrutura de dados atual. O material foi escrito sob a perspectiva do cliente: cada requisito descreve uma necessidade de negócio que o produto deve atender ou uma condição de qualidade que deve ser preservada.

O objetivo da VINUM é oferecer rastreabilidade para vinhos, desde a vinícola, a uva e a safra até o lote produzido, disponibilizando ao cliente uma consulta pública por catálogo e QR Code. A solução possui área administrativa, área do cliente, API, banco MySQL e preparação para uma futura integração com blockchain.

Escopo desta versão: cadastro e relacionamento de vinícolas, tipos de vinho, uvas, vinhos, safras e lotes; autenticação; perfil do cliente; catálogo público; consulta de lote; imagens; QR Code; mensagens de validação; persistência no MySQL com Prisma.

Limite conhecido: a blockchain real ainda não está integrada. A interface, o status e os campos de referência existem como preparação para uma etapa futura, mas o registro imutável em uma rede externa ainda depende de definição técnica e credenciais do provedor.

2. Stakeholders e atores

2.1 Atores e responsabilidades

Código	Requisito	Critério de aceitação / observação
Cliente	Pessoa que consulta o catálogo, visualiza detalhes de vinhos, consulta lotes por QR Code e mantém o próprio perfil.	Não administra dados de produção.
Administrador	Responsável pelo controle completo dos cadastros, relacionamentos, publicação e gestão dos registros.	Possui acesso aos módulos administrativos protegidos.
Editor administrativo	Usuário autorizado a executar operações administrativas de catálogo e rastreabilidade.	Permissões definidas pelo backend.
Vinícola	Organização e origem dos vinhos cadastrados na plataforma.	Mantém dados cadastrais e pode possuir diversos vinhos.
Sistema VINUM	Conjunto formado pelo frontend React, API Express, Prisma ORM e banco MySQL.	Coordena validações, persistência, autenticação e consulta pública.
Futura rede blockchain	Componente externo que poderá registrar a referência imutável do lote.	Fora do escopo implementado em vnum-v1.3.

3. Requisitos funcionais

Os requisitos funcionais descrevem serviços, regras e comportamentos que a aplicação deve oferecer. O critério de aceitação foi escrito para facilitar a validação pelo cliente e, posteriormente, a transformação em casos de teste.

3.1 Acesso, usuários e cadastros de referência

Código	Requisito	Critério de aceitação / observação
RF - 01	A aplicação deve apresentar uma página inicial institucional da VINUM, com acesso ao catálogo de vinhos e às áreas de autenticação.	O visitante consegue iniciar a navegação, conhecer a proposta do sistema e acessar o catálogo ou o login.
RF - 02	A aplicação deve permitir que uma pessoa crie uma conta de cliente informando nome, e-mail e senha.	O e-mail deve ser único; após o cadastro, a conta recebe o perfil CUSTOMER e pode acessar o catálogo.
RF - 03	A aplicação deve permitir login, logout e recuperação da sessão autenticada.	Credenciais válidas iniciam uma sessão; credenciais inválidas recebem mensagem compreensível; o logout encerra o acesso.
RF - 04	O cliente deve poder consultar e atualizar seu perfil, incluindo nome, idade, data de nascimento, endereço, número, cidade, estado, país, telefone e senha.	O formulário valida os dados e mantém o perfil atualizado no banco sem expor a senha.
RF - 05	A aplicação deve controlar o acesso conforme o perfil do usuário: administrador, editor administrativo e cliente.	Cliente acessa catálogo e perfil; administrador/editor acessam a área administrativa; rotas protegidas rejeitam usuários sem permissão.
RF - 06	O administrador ou editor deve poder cadastrar, listar, editar e excluir vinícolas.	O cadastro deve validar nome, CNPJ, cidade, estado e e-mail; o CNPJ deve ser único.
RF - 07	O administrador ou editor deve poder manter os tipos de vinho disponíveis para seleção.	Cada tipo possui nome, descrição e situação ativa/inativa; o nome não pode duplicar outro tipo.
RF - 08	O administrador ou editor deve poder manter o cadastro de uvas utilizadas pelo sistema.	Cada uva possui nome, descrição e situação ativa/inativa; a uva pode ser usada em vinhos, safras e lotes.
RF - 09	O usuário autorizado deve poder cadastrar, consultar, editar e excluir vinhos.	O vinho deve conter nome, tipo, uma ou mais uvas, volume, teor alcoólico, descrição, status e informações complementares.
RF - 10	O cadastro do vinho deve armazenar características, aromas, notas de degustação, harmonização e demais informações destinadas ao cliente.	As informações aparecem na página pública de detalhes do vinho, com textos separados por finalidade.
RF - 11	O usuário autorizado deve poder enviar uma imagem do vinho e visualizar a imagem cadastrada.	A API aceita imagem compatível até o limite definido e vincula a imagem ao vinho correspondente.
RF - 12	O sistema deve permitir relacionar um vinho a um tipo de vinho e a uma composição de uma ou mais uvas.	O salvamento deve impedir vinho sem tipo ou sem pelo menos uma uva válida.

3.2 Safras, lotes e rastreabilidade

Código	Requisito	Critério de aceitação / observação
RF - 13	O administrador ou editor deve poder cadastrar, listar, editar e excluir safras.	A safra deve registrar identificador, ano, origem/fornecedor, observações e situação.
RF - 14	O identificador da safra deve seguir o padrão SF + ano de dois dígitos + código do tanque ou origem.	Ao informar, por exemplo, SF22-T04, o campo ano deve ser preenchido automaticamente como 2022 e o identificador deve ser validado.
RF - 15	A safra deve poder ser relacionada ao vinho correspondente e aos registros de uvas da colheita.	A procedência e a composição da safra devem ficar disponíveis para consulta de rastreabilidade.
RF - 16	O sistema deve impedir a exclusão de uma safra que possua lotes vinculados sem antes resolver os vínculos.	Ao tentar excluir uma safra relacionada, a aplicação deve explicar que existem lotes dependentes.
RF - 17	O administrador ou editor deve poder cadastrar, listar, editar e excluir lotes.	O lote deve reunir código, vinho, safra, uvas utilizadas, quantidade, data de produção, status, data de registro e QR Code.
RF - 18	O código do lote deve seguir o padrão L + ano do envase com dois dígitos + dia do ano com três dígitos.	Um código como L24100 deve ser aceito e formatos incompletos ou inválidos devem ser recusados.
RF - 19	A data de produção do lote deve ser calculada automaticamente a partir do ano e do dia do ano presentes no código.	O sistema deve reconhecer anos bissextos; L24100 corresponde a 09/04/2024 e o campo calculado não deve exigir digitação manual.
RF - 20	O lote deve permitir informar quantidade produzida e relacionar o vinho, a safra e uma ou mais uvas.	O sistema deve exigir quantidade positiva e relações válidas antes de salvar.
RF - 21	O lote deve possuir situações que indiquem seu estágio: aguardando registro, registrado na blockchain ou publicado para consulta.	A situação exibida deve ser intuitiva e a mudança de status deve ser persistida.
RF - 22	A data de registro do lote deve ser preenchida automaticamente quando o lote for salvo com o status Registrado na blockchain.	O campo é somente leitura para o usuário; o backend registra a data e preserva o valor ao editar o lote.
RF - 23	A aplicação deve permitir gerar um QR Code exclusivo para um lote já salvo.	Se o lote ainda não existir, o botão informa que é necessário salvá-lo antes; após a geração, o QR Code fica visível e vinculado ao lote.
RF - 24	O QR Code deve direcionar para a consulta pública do lote pelo código de rastreabilidade.	A rota pública deve apresentar os dados do lote, vinho, safra, procedência, uvas, datas, quantidade e status.

3.3 Catálogo, qualidade e evolução

Código	Requisito	Critério de aceitação / observação
RF - 25	O catálogo público deve listar somente vinhos publicados ou disponíveis para consulta pública.	O cliente não deve visualizar rascunhos ou registros administrativos que não tenham sido publicados.
RF - 26	O catálogo deve permitir pesquisar vinhos e filtrar por tipo.	A opção Todos deve mostrar todos os resultados; os demais tipos devem restringir a lista ao tipo selecionado.
RF - 27	O cliente deve poder abrir a página detalhada de um vinho.	A página deve exibir imagem, tipo, uvas, volume, teor alcoólico, descrição, características, aromas, degustação, harmonização, safras e procedência.
RF - 28	A página detalhada do vinho deve apresentar safras relacionadas quando existirem.	Na ausência de safras publicadas, o sistema deve informar isso de maneira clara, sem erro interno.
RF - 29	As telas administrativas devem permitir pesquisar registros, abrir detalhes, editar e excluir quando o perfil tiver autorização.	A listagem deve atualizar após salvar ou excluir e deve apresentar mensagens de confirmação ou erro.
RF - 30	A aplicação deve preservar rascunhos preenchidos nos formulários quando o usuário trocar de aba ou página antes de salvar.	Ao retornar ao formulário, os dados digitados devem ser recuperados; limpar ou cancelar deve remover o rascunho correspondente.
RF - 31	A API deve disponibilizar rotas de saúde e documentação técnica.	O endpoint de health check deve indicar a disponibilidade do serviço e a documentação OpenAPI deve estar acessível.
RF - 32	A aplicação deve persistir os dados de negócio no MySQL por meio do Prisma ORM.	Criações, alterações e exclusões devem refletir nas tabelas do banco, respeitando chaves, índices e relacionamentos.
RF - 33	A aplicação deve oferecer mensagens intuitivas para erros de validação, duplicidade, vínculos, campos longos e falhas de banco.	O cliente deve receber uma orientação em português sobre o que corrigir, sem depender do código técnico do erro.
RF - 34	A aplicação deve inicializar os perfis essenciais e manter uma conta administrativa de demonstração por seed.	O processo de preparação do ambiente deve criar os papéis e o administrador de demonstração sem expor credenciais no código público de produção.
RF - 35	A aplicação deve manter uma estrutura preparada para uma futura integração com blockchain.	A interface deve explicar que a integração ainda não está disponível; os campos de referência podem permanecer preparados para a próxima etapa.

4. Requisitos não funcionais

Os requisitos não funcionais definem como a solução deve operar. Eles são importantes para que a aplicação não apenas execute suas funções, mas também seja segura, compreensível, consistente, manutenível e adequada ao uso acadêmico e profissional.

4.1 Qualidade, segurança e desenvolvimento

Código	Requisito	Critério de aceitação / observação
RNF - 01	Usabilidade: as telas devem usar linguagem em português, rótulos objetivos, instruções próximas aos campos e mensagens de erro acionáveis.	Uma pessoa que não conhece o banco deve entender o que informar e como corrigir um erro.
RNF - 02	Responsividade: a interface deve se adaptar a computadores, tablets e celulares sem perder campos, botões ou informações importantes.	Formulários, tabelas, catálogo e QR Code devem permanecer utilizáveis em larguras menores.
RNF - 03	Acessibilidade: campos devem ter rótulos, foco visível, textos alternativos em imagens e controles operáveis por teclado.	Os principais fluxos devem ser navegáveis sem depender somente de mouse ou cor.
RNF - 04	Desempenho: consultas do catálogo e das listagens administrativas devem responder sem carregar dados desnecessários.	As buscas devem consultar apenas os campos necessários e utilizar paginação ou filtros quando o volume crescer.
RNF - 05	Segurança de autenticação: senhas não devem ser armazenadas em texto puro e as sessões devem utilizar tokens protegidos.	O backend deve comparar hashes, expirar sessões e invalidar sessões encerradas ou expiradas.
RNF - 06	Autorização: toda operação administrativa deve ser protegida também no backend, não somente escondida no frontend.	Um cliente que tente acessar uma rota administrativa deve receber resposta de acesso negado.
RNF - 07	Proteção de dados: informações de perfil e credenciais devem ser tratadas com confidencialidade e não devem ser publicadas no repositório.	Segredos, senhas e arquivo .env devem permanecer fora do controle de versão.
RNF - 08	Integridade referencial: o banco deve utilizar chaves estrangeiras, unicidade e regras de exclusão coerentes.	Não deve ser possível criar vínculos com registros inexistentes nem apagar dados essenciais sem tratar dependências.
RNF - 09	Persistência: o MySQL deve ser a fonte de dados da aplicação, com o Prisma ORM representando o modelo físico.	O schema Prisma, migrations e configuração devem permanecer sincronizados com o banco.
RNF - 10	Manutenibilidade: o código deve permanecer modular, com frontend, API, autenticação, catálogo e módulos de domínio separados.	Uma alteração em um módulo não deve exigir duplicação da regra em todas as telas.
RNF - 11	Tipagem e qualidade: o projeto deve ser escrito em TypeScript e passar pelas verificações de lint e typecheck.	Os comandos npm run lint e npm run typecheck devem concluir sem erros antes de uma versão.
RNF - 12	Testabilidade: regras importantes de API, autenticação, validação e persistência devem ser cobertas por testes automatizados.	Os testes devem detectar regressões em rotas e regras que protegem o banco.

4.2 Operação, rastreabilidade e evolução

Código	Requisito	Critério de aceitação / observação
RNF - 13	Compatibilidade: a aplicação deve funcionar em navegadores modernos com suporte a React, módulos ES e recursos de formulário usados no projeto.	Chrome, Edge e Firefox atuais devem renderizar os fluxos principais sem comportamento quebrado.
RNF - 14	Configuração por ambiente: URLs, porta, conexão do banco e chave JWT devem ser configuráveis por variáveis de ambiente.	O código não deve depender de credenciais fixas para rodar em desenvolvimento, homologação ou produção.
RNF - 15	Observabilidade: falhas inesperadas devem ser registradas no backend para investigação, enquanto o usuário recebe uma mensagem segura.	A resposta pública não deve expor stack trace, caminho de arquivo ou detalhes internos do banco.
RNF - 16	Validação de entrada: dados vindos do navegador devem ser validados no backend, mesmo que já tenham sido validados no frontend.	Campos obrigatórios, limites, formatos de código, números e datas inválidas devem ser rejeitados de forma consistente.
RNF - 17	Padronização de identificadores: lote e safra devem seguir máscaras e regras determinísticas para facilitar rastreabilidade e leitura humana.	Os formatos L24100 e SF22-T04 devem ser reconhecidos de forma igual em todas as telas e rotas.
RNF - 18	Consistência temporal: datas exibidas ao usuário devem seguir o padrão brasileiro e a data automática do lote deve usar uma regra de fuso definida.	O valor armazenado e o valor apresentado não devem mudar por causa do navegador ou do formato regional.
RNF - 19	Arquivos: uploads devem possuir formatos permitidos, limite de tamanho e armazenamento separado dos arquivos de código.	Imagens acima do limite ou em formato não aceito devem ser recusadas com mensagem clara.
RNF - 20	QR Code: cada código gerado deve ser associado a um lote específico e não deve apontar para dados de outro lote.	A leitura do código deve abrir a rota correta e apresentar o registro correspondente.
RNF - 21	Evolução: a arquitetura deve permitir incluir blockchain real, auditoria e novos campos sem quebrar o catálogo existente.	As referências de blockchain e as migrations devem poder evoluir de forma versionada.
RNF - 22	Disponibilidade: o backend deve possuir health check e o frontend deve tratar estados de carregamento, vazio e erro.	O usuário não deve ficar diante de uma tela sem feedback durante uma consulta.
RNF - 23	Documentação: o projeto deve manter README, documentação da API e instruções de banco, instalação e execução.	Um novo desenvolvedor deve conseguir preparar o ambiente seguindo a documentação.
RNF - 24	Versionamento: alterações de código e banco devem ser registradas em commits identificáveis e versões publicadas no GitHub.	Cada entrega deve permitir identificar o estado do código e repetir a preparação do banco.

5. Regras de negócio consolidadas

As regras abaixo complementam os requisitos e representam decisões que devem ser preservadas durante a implementação.

5.1 Regras de negócio

Código	Requisito	Critério de aceitação / observação
RN - 01	Um vinho deve possuir tipo e pelo menos uma uva.	A composição é obrigatória para a rastreabilidade do produto.
RN - 02	Um lote deve possuir vinho, safra e pelo menos uma uva.	O lote é a unidade de produção consultada pelo QR Code.
RN - 03	O código L + AA + DDD determina a data de produção.	O dia DDD é contado a partir de 1 de janeiro e respeita ano bissexto.
RN - 04	O identificador SF + AA + TDD determina o ano da safra.	O ano completo é derivado dos dois dígitos informados.
RN - 05	A data de registro do lote não é digitada manualmente.	O backend gera a data quando o status selecionado é Registrado na blockchain.
RN - 06	QR Code somente pode ser gerado após salvar o lote.	A geração precisa de um identificador persistido para formar a URL pública.
RN - 07	Uma safra com lotes vinculados não deve ser excluída diretamente.	A integridade da rastreabilidade deve ser preservada.
RN - 08	O catálogo mostra dados destinados à consulta pública.	Informações administrativas e registros não publicados não devem aparecer.
RN - 09	E-mail, CNPJ, identificador de safra, código de lote e slug devem respeitar unicidade quando definidos no modelo.	Duplicidades devem gerar mensagem orientando a correção.
RN - 10	A blockchain real é uma evolução posterior.	A versão atual informa a indisponibilidade sem simular uma confirmação externa.

6. Modelo de dados e integrações

O banco de dados utiliza MySQL e é representado pelo Prisma ORM. As entidades principais são usuario, role, sessao, vinicola, vinho, tipo_vinho, uva, vinho_uva, imagem_vinho, safra, safra_uva, lote, lote_uva e seus status. As tabelas associativas resolvem as composições de muitas uvas para um vinho, uma safra ou um lote.

Relacionamentos centrais: vinícola 1:N vinho; vinho 1:N safra; vinho 1:N lote; safra 1:N lote; vinho N:N uva; safra N:N uva; lote N:N uva; usuário 1:N sessão; vinho 1:1 imagem principal.

O frontend consome a API HTTP. A API aplica autenticação, autorização, validação Zod, regras de negócio e transações Prisma. Uploads de imagem e QR Code são tratados como arquivos separados dos dados relacionais, enquanto o banco armazena suas referências.

7. Matriz resumida de rastreabilidade

7.1 Correspondência entre objetivo e requisitos

Código	Requisito	Critério de aceitação / observação
Cadastro	RF - 06 a RF - 17	Tabelas vinicola, vinho, tipo_vinho, uva, safra e tabelas associativas.
Lote	RF - 17 a RF - 24	Tabela lote, relacionamentos com vinho/safra/uva, status, data e QR Code.
Cliente	RF - 01 a RF - 05 e RF - 25 a RF - 28	Tabelas usuario, role, sessao e consultas públicas.
Qualidade	RNF - 01 a RNF - 24	Frontend responsivo, API protegida, MySQL, Prisma, testes e documentação.
Evolução	RF - 35 e RNF - 21	Campos de referência e arquitetura preparada para blockchain.

8. Critérios de aceite da versão

A versão pode ser considerada pronta para avaliação quando os fluxos abaixo forem executados sem perda de dados ou mensagens técnicas indevidas:

- Criar uma conta de cliente, entrar no sistema, atualizar o perfil e sair.
- Acessar a área administrativa com o perfil autorizado e confirmar que um cliente não consegue acessar as mesmas rotas.
- Cadastrar tipo de vinho, uva, vinho com composição, imagem e informações de degustação.
- Cadastrar safra usando um identificador como SF22-T04 e confirmar o preenchimento automático do ano.
- Cadastrar lote usando um código como L24100 e confirmar a data de produção 09/04/2024.
- Salvar um lote como aguardando registro, gerar o QR Code e abrir a consulta pública.
- Salvar um lote como registrado na blockchain e confirmar a data automática de registro.
- Abrir a página do vinho e confirmar a exibição de safras, procedência e uvas quando houver dados publicados.
- Trocar de aba antes de salvar um formulário e confirmar a recuperação do rascunho.
- Tentar cadastrar valores duplicados, texto acima do limite e relações inexistentes, confirmando mensagens intuitivas.

9. Pendências e próximos levantamentos

Os pontos abaixo não impedem a avaliação da versão atual, mas precisam de decisão do cliente/professor antes de uma versão de produção ou da redação final do TCC:

9.1 Decisões ainda necessárias

Código	Requisito	Critério de aceitação / observação
P - 01	Definir qual rede blockchain será utilizada, custos, carteira, contrato e formato da transação.	Necessário para transformar o botão de preparação em integração real.
P - 02	Definir se a hora de envase ficará visível no formulário do lote ou somente armazenada para uma etapa posterior.	A estrutura de dados possui suporte, mas o campo pode ser ajustado conforme o ER aprovado.
P - 03	Definir política de publicação: quais status tornam vinho, safra e lote visíveis ao cliente.	Evita divergência entre cadastro administrativo e catálogo público.
P - 04	Definir política de backup, retenção de imagens, QR Codes e dados do MySQL.	Necessário para operação contínua e recuperação de incidentes.
P - 05	Validar campos, nomes e cardinalidades do diagrama ER com o professor.	O modelo deve ser a referência para novas migrations e para o artigo.

10. Encerramento

Este levantamento transforma os fluxos atuais da VINUM em uma base formal para desenvolvimento, testes e documentação acadêmica. A numeração RF e RNF pode ser usada no artigo do TCC, no plano de testes e nas futuras discussões de escopo. Quando uma nova funcionalidade for aprovada, ela deve receber um novo identificador ou atualizar explicitamente o requisito relacionado.

Documento preparado para: validação do cliente, alinhamento com o professor orientador e planejamento das próximas versões da aplicação VINUM.